SOFTWARE TESTING LAB

EXAM CHEAT SHEET

All 5 Scenarios | Black Box + White Box + Selenium (Python)

── QUICK SETUP (Run Once) ──

```
pip install selenium
pip install webdriver-manager
# Download chromedriver from https://chromedriver.chromium.org/
# Place chromedriver.exe in same folder as your test scripts
# For auto-install: pip install webdriver-manager
# Then in script: from webdriver_manager.chrome import ChromeDriverManager
#                  driver = webdriver.Chrome(ChromeDriverManager().install())
```

── RUN TESTS ──

```
# 1. Save HTML file (e.g., hms.html) in a folder
# 2. Save test script (e.g., test_hms.py) in SAME folder
# 3. Open terminal in that folder and run:
python test_hms.py
# PASS/FAIL results will print to terminal
```

SCENARIO 1: Hostel Management System (HMS)

1A. BLACK BOX TESTING — Decision Table

Decision Table: Room Allotment Rules

Condition / Rule	R1	R2	R3	R4	R5
CGPA valid (0-10)?	N	Y	Y	Y	Y
CGPA >= 8.0?	-	-	Y	N	N
CGPA >= 6.0?	-	-	-	Y	N
Room available?	-	N	Y	Y	Y
ACTION	—	—	—	—	—
Allot Priority room	✗	✗	✓	✗	✗
Allot Standard room	✗	✗	✗	✓	✗
Show 'No rooms'	✗	✓	✗	✗	✗
Show 'Ineligible'	✗	✗	✗	✗	✓
Show 'Invalid input'	✓	✗	✗	✗	✗

Test Cases from Decision Table:

TC#	CGPA	Available?	Expected Output
TC1	15 (invalid)	—	Invalid CGPA
TC2	8.5	No	No rooms available
TC3	9.0	Yes	Priority allotment – Single AC
TC4	7.0	Yes	Standard allotment – Double Non-AC
TC5	4.0	Yes	Ineligible – CGPA too low

1B. WHITE BOX TESTING — Source Code + Coverage

Source Code (Python):

```
# hms_logic.py (White Box - HMS)
def allot_room(cgpa, room_type, ac_type, available):
    """
    Paths: 1) Invalid CGPA 2) Not available 3) Priority 4) Standard 5) Ineligible
    """
    if cgpa < 0 or cgpa > 10:      # Branch 1
        return "Invalid CGPA"
    if not available:               # Branch 2
        return "No rooms available"
    if cgpa >= 8.0:                 # Branch 3
        return f"Priority: {room_type} {ac_type}"
```

```
    elif cgpa >= 6.0:                # Branch 4
        return f"Standard: {room_type} {ac_type}"
    else:                            # Branch 5
        return "Ineligible"

# Unit Tests (covers all branches)
print(allot_room(-1, "single", "ac", True))   # Branch 1
print(allot_room(7.0, "single", "ac", False)) # Branch 2
print(allot_room(9.0, "single", "ac", True))  # Branch 3
print(allot_room(7.0, "double", "nonac", True)) # Branch 4
print(allot_room(5.0, "single", "ac", True))  # Branch 5
```

Coverage: All 5 branches covered (Statement + Branch + Path coverage = 100%)

1C. SELENIUM AUTOMATION — HTML + Test Script

HTML File (save as hms.html):

```
<!-- hms.html -->
<!DOCTYPE html><html><head><title>HMS</title></head><body>
<h2>Hostel Management System</h2>
<form id="roomForm">
  <label>Room Type:</label>
  <select id="roomType">
    <option value="single">Single</option>
    <option value="double">Double</option>
  </select><br>
  <label>AC:</label>
  <select id="acType">
    <option value="ac">AC</option>
    <option value="nonac">Non-AC</option>
  </select><br>
  <label>CGPA:</label>
  <input type="number" id="cgpa" step="0.01" min="0" max="10"><br>
  <button type="button" onclick="checkAvailability()">Check Room</button>
</form>
<div id="result"></div>
<script>
function checkAvailability() {
  var cgpa = parseFloat(document.getElementById('cgpa').value);
  var room = document.getElementById('roomType').value;
  var ac = document.getElementById('acType').value;
  var msg = '';
  if (isNaN(cgpa) || cgpa < 0 || cgpa > 10) {
    msg = 'Invalid CGPA';
  } else if (cgpa >= 8.0) {
    msg = 'Eligible: Priority Allotment - ' + room + ' ' + ac;
  } else if (cgpa >= 6.0) {
    msg = 'Eligible: Standard Allotment - ' + room + ' ' + ac;
  } else {
    msg = 'Not Eligible: CGPA too low';
  }
  document.getElementById('result').innerHTML = '<p id="resultText">' + msg + '</p>';
}
</script></body></html>
```

Test Script (save as test_hms.py in same folder):

```
# test_hms.py
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select
```

```python
import time, os

driver = webdriver.Chrome()
path = "file://" + os.path.abspath("hms.html")
driver.get(path)
time.sleep(1)

def test_room(cgpa_val, room, ac, expected):
    driver.find_element(By.ID, "cgpa").clear()
    driver.find_element(By.ID, "cgpa").send_keys(str(cgpa_val))
    Select(driver.find_element(By.ID, "roomType")).select_by_value(room)
    Select(driver.find_element(By.ID, "acType")).select_by_value(ac)
    driver.find_element(By.XPATH, "//button").click()
    time.sleep(0.5)
    result = driver.find_element(By.ID, "resultText").text
    status = "PASS" if expected in result else "FAIL"
    print(f"CGPA={cgpa_val} | {status} | Got: {result}")

test_room(9.0, "single", "ac", "Priority")
test_room(7.0, "double", "nonac", "Standard")
test_room(4.0, "single", "ac", "Not Eligible")
test_room(-1, "single", "ac", "Invalid")

driver.quit()
```

How to Run:

- Save hms.html and test_hms.py in the same folder
- Open terminal in that folder
- Run: python test_hms.py
- Output shows PASS/FAIL for each test case

SCENARIO 2: Courseware Management System

2A. BLACK BOX TESTING — Boundary Value Analysis (BVA)

BVA identifies test cases at boundaries of valid input ranges.

Variable	Min	Min+1	Typical	Max-1	Max	Below Min	Above Max
Duration (weeks)	1	2	26	51	52	0	53
No. of Topics	1	2	50	99	100	0	101
Lecturer Hrs/wk	1	2	20	39	40	0	41

Test Cases:

TC#	Duration	Topics	Hrs/wk	Expected
TC1 (Min)	1	1	1	Assigned
TC2 (Min+1)	2	2	2	Assigned
TC3 (Max-1)	51	99	39	Assigned
TC4 (Max)	52	100	40	Assigned
TC5 (Below)	0	5	10	Error: Invalid input
TC6 (Above)	53	5	10	Error: Invalid input
TC7 (Normal)	16	30	20	Assigned

2B. WHITE BOX TESTING — Source Code + Coverage

```
# courseware_logic.py (White Box - Courseware)
def assign_lecturer(duration, topics, hours):
    """
    Paths: 1) Invalid duration 2) Invalid topics 3) Invalid hours 4) Assigned
    """
    if duration < 1 or duration > 52:   # Path 1
        return "Error: duration"
    if topics < 1 or topics > 100:      # Path 2
        return "Error: topics"
    if hours < 1 or hours > 40:         # Path 3
        return "Error: hours"
    return f"Assigned: {duration}wk/{topics}topics/{hours}hrs"  # Path 4

print(assign_lecturer(0, 10, 10))    # Path 1
print(assign_lecturer(12, 0, 10))    # Path 2
print(assign_lecturer(12, 10, 0))    # Path 3
print(assign_lecturer(12, 10, 10))   # Path 4
```

Cyclomatic Complexity: V(G) = 4 (3 if-conditions + 1 base) | All paths covered.

2C. SELENIUM AUTOMATION — HTML + Test Script

HTML File (save as courseware.html):

```html
<!-- courseware.html -->
<!DOCTYPE html><html><head><title>Courseware</title></head><body>
<h2>Courseware System</h2>
<label>Course Duration (weeks):</label>
<input type="number" id="duration" min="1" max="52"><br>
<label>Number of Topics:</label>
<input type="number" id="topics" min="1" max="100"><br>
<label>Lecturer Availability (hrs/week):</label>
<input type="number" id="hours" min="1" max="40"><br>
<button onclick="assign()">Assign Lecturer</button>
<p id="res"></p>
<script>
function assign() {
  var d = parseInt(document.getElementById('duration').value);
  var t = parseInt(document.getElementById('topics').value);
  var h = parseInt(document.getElementById('hours').value);
  if (d < 1 || d > 52 || t < 1 || t > 100 || h < 1 || h > 40) {
    document.getElementById('res').innerText = 'Error: Invalid input';
  } else {
    document.getElementById('res').innerText = 'Assigned: ' + d + 'wk/' + t + 'topics/' + h + 'hrs';
  }
}
</script></body></html>
```

Test Script (save as test_courseware.py):

```python
# test_courseware.py
from selenium import webdriver
from selenium.webdriver.common.by import By
import time, os

driver = webdriver.Chrome()
driver.get("file://" + os.path.abspath("courseware.html"))
time.sleep(1)

def test_course(dur, topics, hrs, expected):
    for fid, val in [("duration", dur), ("topics", topics), ("hours", hrs)]:
        driver.find_element(By.ID, fid).clear()
        driver.find_element(By.ID, fid).send_keys(str(val))
    driver.find_element(By.TAG_NAME, "button").click()
    time.sleep(0.5)
    res = driver.find_element(By.ID, "res").text
    print("PASS" if expected in res else "FAIL", "| Got:", res)

# BVA Tests
test_course(1,  1,  1,  "Assigned")   # Min boundary
test_course(2,  2,  2,  "Assigned")   # Min+1
test_course(52, 100, 40, "Assigned")  # Max boundary
test_course(51, 99, 39, "Assigned")   # Max-1
test_course(0,  5,  10, "Error")      # Below min
test_course(53, 5,  10, "Error")      # Above max

driver.quit()
```

- Save both files in same folder, run: python test_courseware.py

SCENARIO 3: Centralized Citizen Portal

3A. BLACK BOX TESTING — State Transition Table

Current State	Event / Input	Next State	Output
LOGGED_OUT	Valid election card (10 digits)	LOGGED_IN	Login successful
LOGGED_OUT	Invalid card (not 10 digits)	LOGGED_OUT	Invalid card number
LOGGED_IN	Select service (e.g. Railway)	SERVICE_SELECTED	Service activated + Ref No.
LOGGED_IN	Submit petition	PETITION_SUBMITTED	Petition ref number issued
LOGGED_IN	Logout	LOGGED_OUT	Session ended
SERVICE_SELECTED	Submit petition	PETITION_SUBMITTED	Petition ref number issued
SERVICE_SELECTED	Logout	LOGGED_OUT	Session ended
PETITION_SUBMITTED	Track petition	PETITION_SUBMITTED	Status shown with ref no.
PETITION_SUBMITTED	Logout	LOGGED_OUT	Session ended

States: LOGGED_OUT → LOGGED_IN → SERVICE_SELECTED → PETITION_SUBMITTED

3B. WHITE BOX TESTING — Source Code + Coverage

```
# citizen_logic.py (White Box - Citizen Portal)
class CitizenPortal:
    def __init__(self):
        self.state = "LOGGED_OUT"
    def login(self, card):
        if len(card) == 10 and card.isdigit():  # Path 1 valid
            self.state = "LOGGED_IN"
            return "Login successful"
        return "Invalid card"  # Path 2 invalid
    def select_service(self, service):
        if self.state != "LOGGED_IN":  # Path 3
            return "Not logged in"
        self.state = "SERVICE_SELECTED"
        return f"Service {service} selected"
    def logout(self):
        self.state = "LOGGED_OUT"
        return "Logged out"

p = CitizenPortal()
print(p.login("1234567890"))   # Path 1
print(p.select_service("ration")) # Path 3 success
print(p.logout())
print(p.select_service("ration")) # Path 3 fail
```

All state transitions tested. Statement coverage: 100%

3C. SELENIUM AUTOMATION — HTML + Test Script

HTML File (save as citizen.html):

```
<!-- citizen.html -->
<!DOCTYPE html><html><head><title>Citizen Portal</title></head><body>
<h2>Citizen Government Portal</h2>
<div id="loginSection">
  <input id="electionCard" placeholder="Election Card No"><br>
  <button onclick="login()">Login</button>
</div>
<div id="services" style="display:none">
  <h3>Services</h3>
  <button onclick="selectService('railway')">Railway Concession</button>
  <button onclick="selectService('hospital')">Hospital Appointment</button>
  <button onclick="selectService('ration')">Ration Card</button>
  <button onclick="submitPetition()">Submit Petition</button>
  <button onclick="logout()">Logout</button>
  <p id="svcResult"></p>
</div>
<p id="loginResult"></p>
<script>
var state = "LOGGED_OUT";
function login() {
  var card = document.getElementById('electionCard').value;
  if (card.length === 10 && /^d+$/.test(card)) {
    state = "LOGGED_IN";
    document.getElementById('services').style.display='block';
    document.getElementById('loginSection').style.display='none';
    document.getElementById('loginResult').innerText = 'Login successful';
  } else { document.getElementById('loginResult').innerText = 'Invalid card'; }
}
function selectService(s) {
  if (state === "LOGGED_IN") {
    state = "SERVICE_SELECTED";
    document.getElementById('svcResult').innerText = 'Service: ' + s + ' | Ref: REF' + Date.now();
  }
}
function submitPetition() {
  if (state === "LOGGED_IN" || state === "SERVICE_SELECTED") {
    document.getElementById('svcResult').innerText = 'Petition submitted. Ref: PET' + Date.now();
  }
}
function logout() {
  state = "LOGGED_OUT";
  document.getElementById('services').style.display='none';
  document.getElementById('loginSection').style.display='block';
  document.getElementById('loginResult').innerText = 'Logged out';
}
</script></body></html>
```

Test Script (save as test_citizen.py):

```
# test_citizen.py
from selenium import webdriver
from selenium.webdriver.common.by import By
import time, os

driver = webdriver.Chrome()
driver.get("file://" + os.path.abspath("citizen.html"))
time.sleep(1)

def test_login(card, expected):
    try: driver.find_element(By.ID, "loginSection").find_element(By.TAG_NAME, "button").click()
    except: pass
    driver.find_element(By.ID, "electionCard").clear()
    driver.find_element(By.ID, "electionCard").send_keys(card)
```

```
    driver.find_element(By.XPATH, "//button[text()='Login']").click()
    time.sleep(0.5)
    res = driver.find_element(By.ID, "loginResult").text
    print("PASS" if expected in res else "FAIL", "| Card:", card, "| Got:", res)

# State Transition Tests
test_login("1234567890", "successful")  # LOGGED_OUT -> LOGGED_IN
# Test service selection
driver.find_element(By.XPATH, "//button[text()='Railway Concession']").click()
time.sleep(0.5)
res = driver.find_element(By.ID, "svcResult").text
print("PASS" if "REF" in res else "FAIL", "| Service selected")
# Test logout
driver.find_element(By.XPATH, "//button[text()='Logout']").click()
time.sleep(0.5)
print("PASS" if "Logged out" in driver.find_element(By.ID, "loginResult").text else "FAIL", "| Logout")
# Test invalid card
test_login("ABC", "Invalid")

driver.quit()
```

- Save both files in same folder, run: python test_citizen.py

SCENARIO 4: Library Information System (LIS)

4A. BLACK BOX TESTING — Equivalence Class Partitioning (ECP)

Input	Valid Class (EC1)	Invalid Class (EC2)	Invalid Class (EC3)
Member ID	M001 – M999 (starts with M, 4 chars)	No M prefix (001, XYZ)	Empty string
Book ID	B001 – B999 (starts with B, 4 chars)	No B prefix (001, XYZ)	Empty string
Availability	Book not borrowed = can borrow	Book already borrowed	—

Test Cases from ECP:

TC#	Member ID	Book ID	Action	Expected
TC1 (Valid)	M001	B001	Borrow	Borrowed B001 by M001
TC2 (Already)	M002	B001	Borrow	Book already borrowed
TC3 (Invalid M)	001	B002	Borrow	Invalid ID format
TC4 (Invalid B)	M003	999	Borrow	Invalid ID format
TC5 (Reserve)	M004	B005	Reserve	Reserved B005 for M004
TC6 (Empty)		B001	Borrow	Invalid ID format

4B. WHITE BOX TESTING — Source Code + Coverage

```
# library_logic.py (White Box - Library)
class Library:
    def __init__(self):
        self.loans = {}
        self.reservations = {}
    def borrow(self, member_id, book_id):
        if not member_id.startswith('M') or not book_id.startswith('B'):
            return "Invalid ID"  # Path 1
        if book_id in self.loans:
            return "Already borrowed"  # Path 2
        self.loans[book_id] = member_id
        return f"Borrowed {book_id}"  # Path 3
    def reserve(self, member_id, book_id):
        self.reservations[book_id] = member_id
        return f"Reserved {book_id}"  # Path 4

lib = Library()
print(lib.borrow("M001", "B001"))  # Path 3
print(lib.borrow("M002", "B001"))  # Path 2
print(lib.borrow("001",  "B002"))  # Path 1
print(lib.reserve("M003","B002"))  # Path 4
```

4 distinct paths covered. Branch coverage: 100%

4C. SELENIUM AUTOMATION — HTML + Test Script

HTML File (save as library.html):

```
<!-- library.html -->
<!DOCTYPE html><html><head><title>Library System</title></head><body>
<h2>Library Information System</h2>
<input id="memberId" placeholder="Member ID (M001-M999)"><br>
<input id="bookId" placeholder="Book ID (B001-B999)"><br>
<button onclick="borrowBook()">Borrow</button>
<button onclick="reserveBook()">Reserve</button>
<p id="loanResult"></p>
<script>
var loans = {};
var reservations = {};
function validateId(id, prefix) {
  return id.startsWith(prefix) && id.length === 4 && !isNaN(id.slice(1));
}
function borrowBook() {
  var mid = document.getElementById('memberId').value;
  var bid = document.getElementById('bookId').value;
  if (!validateId(mid,'M') || !validateId(bid,'B')) {
    document.getElementById('loanResult').innerText = 'Invalid ID format';
    return;
  }
  if (loans[bid]) {
    document.getElementById('loanResult').innerText = 'Book already borrowed';
  } else {
    loans[bid] = mid;
    document.getElementById('loanResult').innerText = 'Borrowed: ' + bid + ' by ' + mid;
  }
}
function reserveBook() {
  var mid = document.getElementById('memberId').value;
  var bid = document.getElementById('bookId').value;
  if (!validateId(mid,'M') || !validateId(bid,'B')) {
    document.getElementById('loanResult').innerText = 'Invalid ID format';
    return;
  }
  reservations[bid] = mid;
  document.getElementById('loanResult').innerText = 'Reserved: ' + bid + ' for ' + mid;
}
</script></body></html>
```

Test Script (save as test_library.py):

```
# test_library.py
from selenium import webdriver
from selenium.webdriver.common.by import By
import time, os

driver = webdriver.Chrome()
driver.get("file://" + os.path.abspath("library.html"))
time.sleep(1)

def test_borrow(mid, bid, expected):
    driver.find_element(By.ID, "memberId").clear()
    driver.find_element(By.ID, "memberId").send_keys(mid)
    driver.find_element(By.ID, "bookId").clear()
    driver.find_element(By.ID, "bookId").send_keys(bid)
    driver.find_element(By.XPATH, "//button[text()='Borrow']").click()
    time.sleep(0.5)
    res = driver.find_element(By.ID, "loanResult").text
```

```
    print("PASS" if expected in res else "FAIL", "| Got:", res)

# ECP: Valid member/book IDs
test_borrow("M001", "B001", "Borrowed")
# ECP: Book already borrowed
test_borrow("M002", "B001", "already")
# ECP: Invalid member ID (no M prefix)
test_borrow("001",  "B002", "Invalid")
# ECP: Invalid book ID
test_borrow("M003", "999",  "Invalid")
# Reserve test
driver.find_element(By.ID, "memberId").clear()
driver.find_element(By.ID, "memberId").send_keys("M004")
driver.find_element(By.ID, "bookId").clear()
driver.find_element(By.ID, "bookId").send_keys("B005")
driver.find_element(By.XPATH, "//button[text()='Reserve']").click()
time.sleep(0.5)
print("PASS" if "Reserved" in driver.find_element(By.ID, "loanResult").text else "FAIL", "| Reserve")

driver.quit()
```

- Save both files in same folder, run: python test_library.py

SCENARIO 5: Online Food Ordering System

5A. BLACK BOX TESTING — Boundary Value Analysis (BVA)

Variable	Min	Min+1	Nominal	Max-1	Max	Below Min	Above Max
Quantity (items)	1	2	5	9	10	0	11
Payment modes	card	netbanking	—	—	cash(pickup)	—	cash+delivery=invalid

Test Cases:

TC#	Restaurant	Qty	Mode	Payment	Expected
TC1 (Min)	Pizza	1	Delivery	Card	Order placed
TC2 (Min+1)	Pizza	2	Delivery	Card	Order placed
TC3 (Max-1)	Burger	9	Delivery	NetBanking	Order placed
TC4 (Max)	Pizza	10	Pickup	Cash	Order placed
TC5 (Below)	Pizza	0	Delivery	Card	Invalid quantity
TC6 (Above)	Pizza	11	Delivery	Card	Invalid quantity
TC7 (No rest)		5	Delivery	Card	Select restaurant
TC8 (Cash+Del)	Pizza	5	Delivery	Cash	Cash only for pickup

5B. WHITE BOX TESTING — Source Code + Coverage

```
# food_logic.py (White Box - Food Ordering)
def place_order(restaurant, qty, delivery_mode, payment_mode):
    if not restaurant:              # Path 1
        return "Select restaurant"
    if qty < 1 or qty > 10:        # Path 2
        return "Invalid quantity"
    if payment_mode == 'cash' and delivery_mode != 'pickup':  # Path 3
        return "Cash only for pickup"
    return f"Ordered from {restaurant} x{qty}"  # Path 4

print(place_order("", 5, "delivery", "card"))        # Path 1
print(place_order("pizza", 0, "delivery", "card"))   # Path 2
print(place_order("pizza", 5, "delivery", "cash"))   # Path 3
print(place_order("pizza", 5, "pickup",   "cash"))   # Path 4
```

4 paths traced. Cyclomatic Complexity = 4. Statement + Branch coverage = 100%

5C. SELENIUM AUTOMATION — HTML + Test Script

HTML File (save as foodorder.html):

```
<!-- foodorder.html -->
<!DOCTYPE html><html><head><title>Food Ordering</title></head><body>
<h2>Online Food Ordering System</h2>
<select id="restaurant">
```

```
  <option value="">-- Select Restaurant --</option>
  <option value="pizza">Pizza Palace</option>
  <option value="burger">Burger Barn</option>
</select><br>
<input id="quantity" type="number" placeholder="Quantity (1-10)" min="1" max="10"><br>
<select id="deliveryMode">
  <option value="delivery">Delivery</option>
  <option value="pickup">Pickup</option>
</select><br>
<select id="paymentMode">
  <option value="card">Credit/Debit Card</option>
  <option value="netbanking">Net Banking</option>
  <option value="cash">Cash (Pickup only)</option>
</select><br>
<button onclick="placeOrder()">Place Order</button>
<p id="orderResult"></p>
<script>
function placeOrder() {
  var rest = document.getElementById('restaurant').value;
  var qty = parseInt(document.getElementById('quantity').value);
  var mode = document.getElementById('deliveryMode').value;
  var pay = document.getElementById('paymentMode').value;
  if (!rest) { document.getElementById('orderResult').innerText='Select restaurant'; return; }
  if (qty < 1 || qty > 10) { document.getElementById('orderResult').innerText='Invalid quantity';
return; }
  if (pay === 'cash' && mode !== 'pickup') {
    document.getElementById('orderResult').innerText='Cash only for pickup';
    return;
  }
  document.getElementById('orderResult').innerText =
    'Order placed! ' + rest + ' x' + qty + ' | ' + mode + ' | ' + pay + ' | Ref: ORD' + Date.now();
}
</script></body></html>
```

Test Script (save as test_foodorder.py):

```
# test_foodorder.py
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select
import time, os

driver = webdriver.Chrome()
driver.get("file://" + os.path.abspath("foodorder.html"))
time.sleep(1)

def test_order(rest, qty, mode, pay, expected):
    Select(driver.find_element(By.ID,"restaurant")).select_by_value(rest) if rest else None
    driver.find_element(By.ID,"quantity").clear()
    driver.find_element(By.ID,"quantity").send_keys(str(qty))
    Select(driver.find_element(By.ID,"deliveryMode")).select_by_value(mode)
    Select(driver.find_element(By.ID,"paymentMode")).select_by_value(pay)
    driver.find_element(By.TAG_NAME,"button").click()
    time.sleep(0.5)
    res = driver.find_element(By.ID,"orderResult").text
    print("PASS" if expected in res else "FAIL", "| qty:", qty, "| Got:", res)

# BVA Tests
test_order("pizza", 1,  "delivery", "card",       "Order placed")  # min qty
test_order("pizza", 2,  "delivery", "card",       "Order placed")  # min+1
test_order("pizza", 10, "pickup",   "cash",       "Order placed")  # max qty
test_order("pizza", 9,  "delivery", "netbanking", "Order placed")  # max-1
test_order("pizza", 0,  "delivery", "card",       "Invalid")       # below min
test_order("pizza", 11, "delivery", "card",       "Invalid")       # above max
```

```
test_order("pizza", 5,  "delivery", "cash",       "Cash only")     # cash+delivery invalid

driver.quit()
```

- Save both files in same folder, run: python test_foodorder.py

QUICK REFERENCE — Testing Methods

Black Box Methods Summary

Method	When to Use	Key Idea
Decision Table (S1)	Multiple conditions → action combos	List all condition combos and expected actions
Boundary Value Analysis (S2,S5)	Numeric ranges as inputs	Test at: min, min+1, max-1, max, below min, above max
State Transition (S3)	System has distinct states	Draw states, transitions, events, expected outputs
Equivalence Class Partitioning (S4)	Group similar inputs	Valid classes (should pass), Invalid classes (should fail)

White Box Testing — Key Terms

Term	Definition
Statement Coverage	Every line of code executed at least once
Branch Coverage	Every if/else branch taken at least once
Path Coverage	Every unique execution path tested
Cyclomatic Complexity	V(G) = Edges - Nodes + 2 × Components (or: # decisions + 1)

Selenium Cheat Sheet (Python)

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select
import time, os

driver = webdriver.Chrome()                      # Open browser
driver.get("file://" + os.path.abspath("x.html"))  # Open local HTML
driver.find_element(By.ID, "inputId")           # Find by ID
driver.find_element(By.ID, "inputId").send_keys("text") # Type
driver.find_element(By.ID, "inputId").clear()  # Clear field
driver.find_element(By.XPATH, "//button").click() # Click button
driver.find_element(By.XPATH, "//button[text()='Login']").click()
Select(driver.find_element(By.ID, "dropdownId")).select_by_value("val")
element.text                                     # Get text of element
time.sleep(0.5)                                  # Wait 0.5 seconds
driver.quit()                                    # Close browser
```

Common Exam Definitions

Term	Definition
Black Box Testing	Testing without knowledge of internal code; based on I/O
White Box Testing	Testing with knowledge of code; covers paths/branches

Term	Definition
Automation Testing	Using tools (Selenium) to run tests automatically
Test Case	Input + expected output + actual output + pass/fail
Test Script	Code that automates test execution
Defect/Bug	Difference between expected and actual behavior
Regression Testing	Re-running tests after code changes to ensure no breaks
Selenium	Open-source tool for automating web browser actions